



MIScan

project documentation

Luka Duplančić, XV Gymnasium, mentor: Ernest Wendling

Zagreb, April 2024

Contents

1.	Introduction	3
2.	User interface.....	4
2.1.	Creating a new photo	4
2.2.	Book page scanning	5
2.3.	A file with the same name already exists	6
2.4.	File deletion	6
2.5.	Gallery export	6
2.6.	Downloading the file in PDF format	7
2.7.	Edge case when straightening a photo.....	7
3.	Technical implementation of the application.....	8
4.	Photo transformation algorithms.....	9
4.1.	Quadrilateral transformation algorithm (see <i>Figures 5 and 6</i>).....	9
4.1.1.	Input and output data.....	9
4.1.2.	One-dimensional case.....	9
4.1.3.	Two-dimensional case.....	11
4.1.4.	Implementation	13
4.2.	Book page straightening algorithm (see <i>Figures 8 and 9</i>)	14
4.2.1.	Input and output data.....	14
4.2.2.	Determining the model curve from the edge curve	15
4.2.3.	Transforming the image	15
4.2.4.	Implementation	16
4.3.	Interaction with external code	17
4.3.1.	Function calling order	17
4.4.	Testing.....	17
5.	Further development plan	18
6.	Additional examples.....	18
		19

1. Introduction

My name is Luka Duplančić. I am a 11th grade in the XV Gymnasium in Zagreb. So far, I have participated in numerous academic competitions, including the Competition in Computer Science in the Application of Algorithms category. I was exclusively involved in programming competitively in the C++ language, but I am also interested in writing programs for practical purposes such as creating mobile applications, so I decided to compete in the Software Development category.

Among family members, I noticed that photographing documents with a mobile phone is a common way to send documentation. Document photography is used instead of scanning. However, unlike scanning on a desktop scanner, handheld photography is more difficult to control and documents are practically always photographed at an angle and are perspective distorted in the photos. I was interested in how to mathematically solve the problem of flattening a photographed document when the image is photographed at an angle, and that's how the idea for creating this application came about.

The algorithm I have developed practically simulates the process of photography, and I use it in the opposite direction in such a way that from a photo of a document, i.e. projection, I "project" the appearance of the original rectangular document. I would like to mention that there are already mobile applications that do the aforementioned correction of photographed documents, but they use a different, more formal approach through the transformation matrix. What ultimately proves to be an advantage of my approach is the ability to easily expand to "scan" books, magazines and other bound material. Namely, if we want to "scan" a page of a book by photographing, the only problem will not be the perspective distortion due to photographing at an angle, but there will also be a problem of bending the sheet of the book, which is a consequence of the binding at one end. Mobile applications that "straighten" photographed pages of books and magazines are not common, and the quality of smoothing the bend varies greatly from example to example, which later bothers us in case we want to do automatic text recognition (OCR). Because of all this, it is necessary to try new ideas and algorithms because it is difficult to recognize in advance which approach will lead to a better result. One such different approach is the algorithm that I designed and implemented in the MIScan mobile application.

MIScan (short for *Mathematically Improved Scanner*) is a mobile application for quick and easy photo straightening. One of the features of the application is its simplicity, there is no need to create a user account, and the straightened photos can be easily shared from the application, stored in the gallery of a mobile device or downloaded in PDF form. The application of the application ranges from quick photography of the necessary documentation for applications to archiving, for example, drawings and notes and scanning magazines, books, etc.

The app supports straightening book pages. This presents an additional challenge because the pages of the books are not as flat as a rectangular sheet of paper. Other applications designed for similar purposes generally do not support this method of scanning (*Figures 7-9*).

The app is supported on *Android* operating systems. The minimum supported Android API level is 19, which is standard for today's *Android* apps. Currently, the application is not supported on *iOS* systems, but it is possible to customize the code of the application so that it is supported on them.

2. User interface

The app is bilingual. Croatian and English language are supported. The Croatian language is used only when the default language of the device is Croatian, otherwise the application interface is always in English. The interface shown here is in Croatian language.

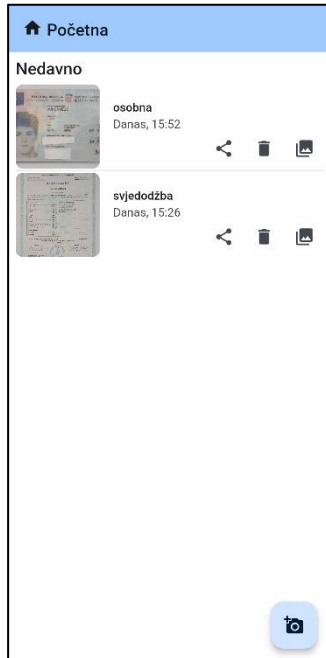


Figure 1: Home page



Figure 2: Photo display page

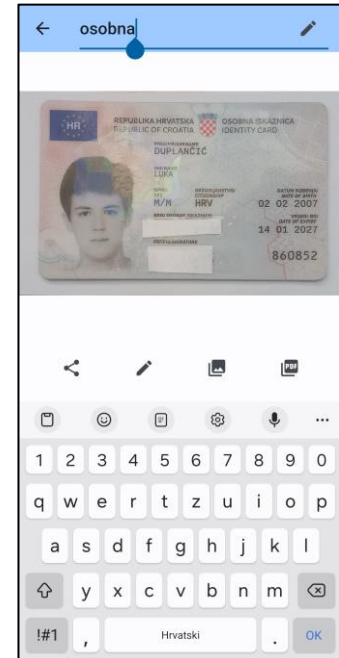


Figure 3: Renaming a photo

The home page of the application (*Figure 1*) contains a list of previously straightened photos sorted by the time of the last modification. Each item from the list can be shared, deleted, exported to the device gallery or downloaded in PDF form. Clicking on an item from the list opens the photo view page (*Figure 2*) where it is possible to see the photo more clearly, rename it (*Figure 3*), share or edit it. Photo editing supports rotating the photo 90 degrees left or right, renaming the photo, and changing the contrast and brightness of the photo.

2.1. Creating a new photo

A new photo can be created by pressing the button at the bottom right of the home page (*Figure 1*). After pressing the button, a pop-up window appears (*Figure 4*) that asks the user to choose whether they want to create the photo using the camera or import it from the gallery. After the user takes a picture or selects a photo, a page opens that asks the user to select the corners of the part of the photo they want to flatten (*Figure 5*). The selection of corners is done using four circles that the user can drag. When dragging a circle, a window appears that shows the part of the photo below the circle that the user is dragging. This helps the user to select the tops of the document more precisely. After selecting the corners, the user starts the process of straightening the desired part of the photo by pressing the checkmark button.

In the next step, the photo editor opens (*Figure 6*), which allows you to rotate 90 degrees to the left or right and change the contrast and brightness of the photo. The default name of the photo is in the form of "Scan_YYYYMMDD_hhmmss", where YYYYMMDD indicates the date and hhmmss the time when the photo was cropped. It is possible to rename a photo in the editor.

After the user finishes editing the photo, clicking on the checkmark saves the photo and opens the photo view page (Figure 2). The saved photo is listed in the homepage (Figure 1).



Figure 4: Pop-up window for selecting the photo source



Figure 5: Document Corner Selection Page

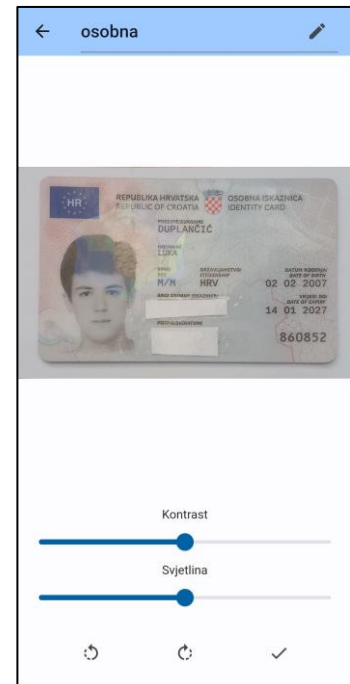


Figure 6: Photo Editor

2.2. Book page scanning

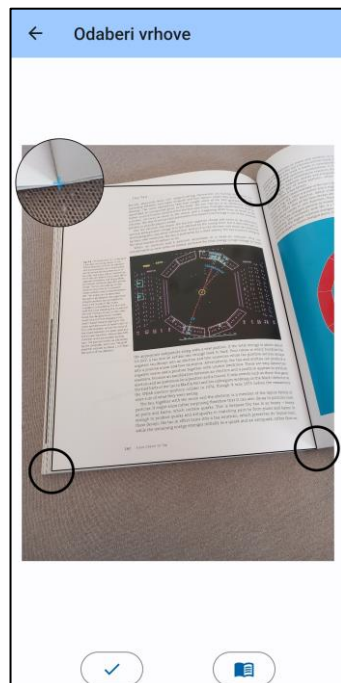


Figure 7: Selecting book page corners

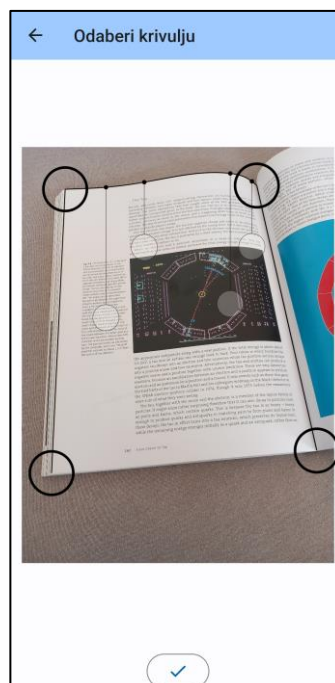


Figure 8: Selecting the curvature of the book page

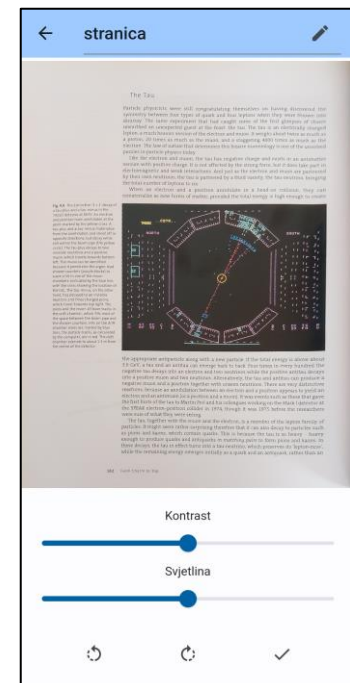


Figure 9: Photo editor with the result of transforming the image of the book page

Straightening a book page poses an additional challenge because the pages of the book are bent due to the binding. That's why I developed my own algorithm to solve this problem. After the user selects the book page tops on the vertex picker page (Figure 7), pressing the book icon button opens the page for selecting the curvature curve of the book page (Figure 8). The user adjusts the shape of the curve using four sliders that can be dragged across the screen. After selecting the curve, pressing the check mark flattens the image and opens the editor, as already described (Figure 9).

2.3. A file with the same name already exists

When you rename a file, export a photo to a gallery, or download a photo as a PDF, it may happen that a file with the same name already exists in the file's save location. In this case, the application will notify the user and ask if the user wants to overwrite the existing file or cancel the action (Figure 10).

2.4. File deletion

When deleting a photo, the application asks the user for confirmation that they really want to delete that photo (Figure 11). This prevents the user from accidentally deleting a photo.

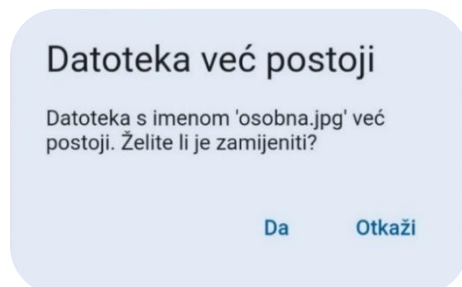


Figure 10: Pop-up to notify the user that the file already exists

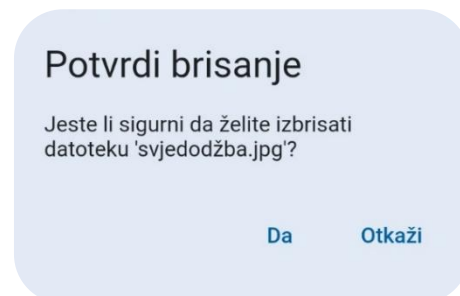


Figure 11: Deletion confirmation pop-up

2.5. Gallery export

When exporting a file to the gallery, the application also asks for user confirmation (Figure 12). The exported files are saved in the MIScan folder in the device gallery. The application notifies the user if a file with the same name as already described already exists in the specified folder. After exporting the file, a notification appears confirming the successful export (Figure 13). The exported files are not associated with the application and can be used separately, opened in the device gallery, edited in the gallery, etc.

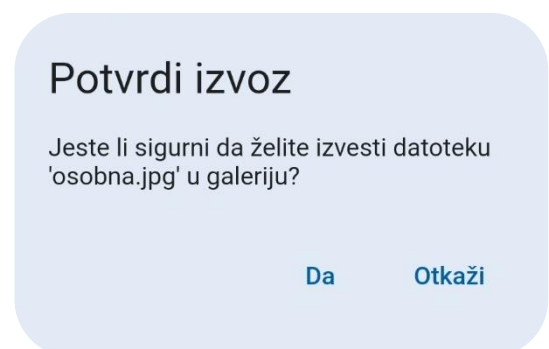


Figure 12: Export Confirmation Pop-up

2.6. Downloading the file in PDF format

Because PDF format is the standard for storing documents, the application supports downloading images in PDF format. By clicking on the button with the PDF icon on the photo view page, the photo is converted to PDF format and saved in the Downloads folder of the mobile device (Figure 14). I chose the Downloads folder as the save location based on a survey I conducted among students at my school.

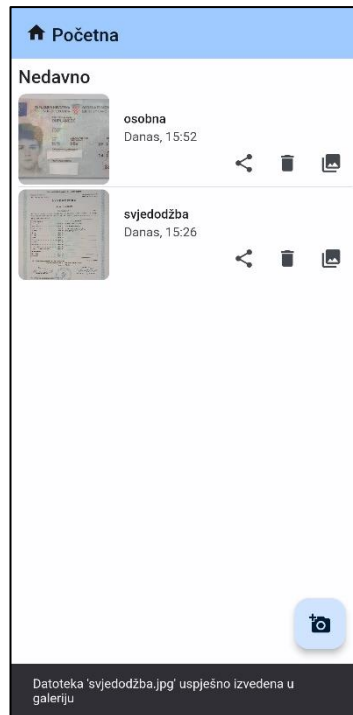


Figure 13: Successful export to gallery notification



Figure 14: Successful PDF document download notification

2.7. Edge case when straightening a photo

It is possible for the user to choose such corners that it is mathematically impossible to uniquely determine the necessary transformation for straightening the photo. Although it is unlikely that the user accidentally guesses such a selection of peaks, especially if the application is used for the purposes for which it is intended, the application will notify the user in this case and the transformation will not be carried out (Figure 15).



Figure 15: Notification that transformation from selected vertices is impossible

3. Technical implementation of the application

The application was created in the *Flutter* framework. *Flutter* uses the *Dart* programming language to build applications. The user interface of the application is programmed in *Dart*, while the photo transformation algorithms are written in *C++* for better time performance because they require processing a larger amount of data. *VS Code* was used to write all of the code.

After the transformation (straightening), the photo is in an uncompressed form, that is, it is displayed in memory as a matrix that contains the color values in that pixel for each pixel of the photo. As such, a photo takes up a lot of memory and needs to be compressed. On mobile devices, the most common compressed photo format is JPEG. Compressing a photo into JPEG format and decompressing it is a computationally demanding process, so existing *C++* code was used for this purpose¹. Copied code files are under *Public Domain license* and contain information about the author and the license.

The code to change the contrast and brightness of a photo was written by me in *C++*.

The selection of the curve when transforming a book page is done using *Natural Cubic Spline* interpolation between the points that the user drags on the screen. The code that performs the interpolation is located in the *lib/cubic_spline.dart* file, and more about the mathematical background of *Cubic Spline* interpolation can be found here <https://blog.timodenk.com/cubic-spline-interpolation/index.html>.

Parts of the code written in *C++* are compiled into a shared library when the application is installed from which *Dart* code can call *C++* functions.

The corresponding translations of all messages that the application displays to the user are located in the *lib/l10n* folder in the *app_en.arb* files for English and *app_hr.arb* for the Croatian language. These files are formatted in *JSON* format and contain the text of the message in the appropriate language for each message key. The *l10n.yaml* file contains instructions for generating *Dart* files with translations from the given *ARB* files that are then used in the application code.

All of the *Dart* code written by me is located in the *lib* folder. A list of libraries used is in the *pubspec.yaml* file under *dependencies*.

The *C++* code is located in the *native* folder. The *native/basic_linear* folder contains implementations of geometric objects such as vector, line, and plane. The *native/transform* folder contains the implementation of photo transformation algorithms that use the objects implemented in the *native/basic_linear* folder. In the *native/jpeg_compressor* folder there is the already described code for compressing the image in JPEG format, and the code in the *native/jpg_encode* folder adapts the specified code so that it can be used in *Dart* code. The code in the *native/straighten* folder adapts the transform code so that it can be used in *Dart* code. The code in the *native/image_processing* folder is used to change the contrast and brightness of the photo.

The *android/app/CMakeLists.txt* file contains instructions for compiling the *C++* code. The code from the *native/basic_linear*, *native/jpeg_compressor* and *native/transform* folders is compiled into respective static libraries used by the code in *native/exports/straighten.cpp*, *native/jpg_encode/jpg_encode.cpp* and *native/image_processing/image_processing.cpp* files which are compiled into respective shared libraries which can be used in *Dart* code.

¹ The code for compressing and decompressing an image in JPEG format is taken from the page <https://github.com/richgel999/jpeg-compressor>

4. Photo transformation algorithms

Photo transformation algorithms are designed exclusively by me. The implementations of the algorithms themselves are original and the only external code used is the *parallel_for²* function, which is used for iterating through all pixels of the photo in parallel and thus speeding up the execution of the program.

4.1. Quadrilateral transformation algorithm (see Figures 5 and 6)

4.1.1. Input and output data

The input data of the algorithm is the image itself that we want to flatten (in the form of a matrix where each field of the matrix represents one pixel of the image) and the coordinates of the vertices of the area we want to flatten (in Figure 17 these are the coordinates of points A , B , C and D)

The output of the algorithm is a flattened image also in the form of a matrix.

4.1.2. One-dimensional case

To describe how the algorithm works, let's first look at the one-dimensional case (Figure 16).

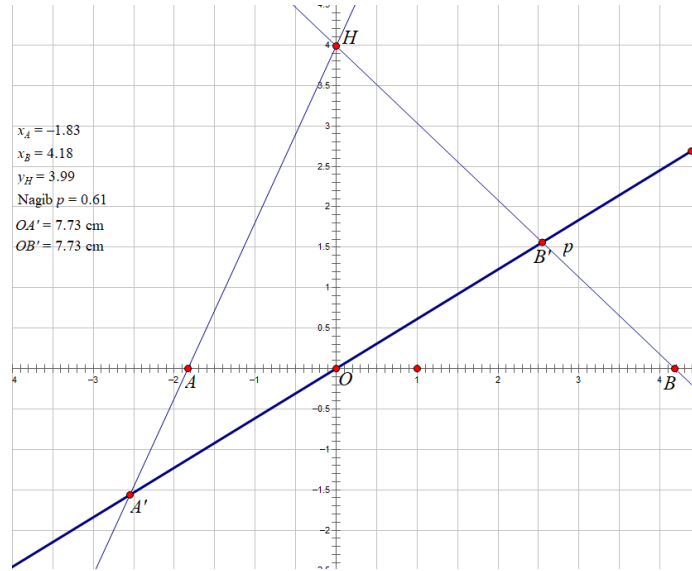


Figure 16: One-dimensional case

In a one-dimensional case, the photographed object is the line segment $\overline{A'B'}$. Let O be the midpoint of the segment $\overline{A'B'}$ of line p . Now let's define the coordinate system S so that O is the origin and the y-axis passes through H (the camera oculus). By photographing the segment $\overline{A'B'}$ with a camera which is located at H we get the image represented with the segment $\overline{AB}$ tag. Knowing the coordinates of points A and B on the x-axis of the system S and the distance of points A' and B' from point O , we can determine the slope of the line p in system S . Let's introduce the following labels:

- a – distance from point A to point O
- b – distance from point B to point O
- l – the distance of points A' and B' from point O , i.e. half the distance from A' to B'
- t – slope of line p .

² The code for the *parallel_for* function is taken from the page <https://stackoverflow.com/a/49188371>

In the one-dimensional case, our algorithm knows the values of a , b and l , and we want to obtain the value of t in order to determine the equation of the line p on which the photographed segment $\overline{A'B'}$ is located, which the camera projected on the x-axis of the system S , into the segment $\overline{AB}$.

We want to express the slope of the line p from the known quantities. With T_x we denote the x coordinate of a point T , and with T_y its y coordinate in the system S .

For the coordinates of the point B' , it holds:

$$\begin{aligned} B_x'^2 + B_y'^2 &= l^2 \\ B_y' &= tB_x' \\ B_x' &= l \sqrt{\frac{1}{1+t^2}}, \\ B_y' &= l \sqrt{\frac{t^2}{1+t^2}}. \end{aligned}$$

Positive or negative roots are taken according to *Figure 16*. For the point A' it analogously holds:

$$\begin{aligned} A_x' &= -l \sqrt{\frac{1}{1+t^2}}, \\ A_y' &= -l \sqrt{\frac{t^2}{1+t^2}}. \end{aligned}$$

Now we can determine the equations of the lines $B'B$ and $A'A$ in the system S . We already know the coordinates of points A' and B' , the coordinates of point A are $(-a, 0)$, and the coordinates of point B are $(b, 0)$. We obtain the following equations of these lines:

$$\begin{aligned} \text{line } A'A: y &= \frac{lt}{l - a\sqrt{1+t^2}}(x + a), \\ \text{line } B'B: y &= \frac{-lt}{b\sqrt{1+t^2} - l}(x - b). \end{aligned}$$

These lines intersect at point H whose x-coordinate is 0. Therefore, by substituting in $x = 0$ in the two equations above we must get the same y . It follows:

$$\begin{aligned} \frac{lt}{l - a\sqrt{1+t^2}}(0 + a) &= \frac{-lt}{b\sqrt{1+t^2} - l}(0 - b), \\ \frac{a}{l - a\sqrt{1+t^2}} &= \frac{b}{b\sqrt{1+t^2} - l}, \\ \sqrt{1+t^2} &= \frac{a+b}{2ab}l, \end{aligned}$$

$$\boxed{t = \pm \sqrt{\left(\frac{a+b}{2ab}l\right)^2 - 1}}. \quad (1)$$

Equation (1) determines the slope of the line p in the system S .

The coordinates of the camera (point H) are of the form $(0, h)$. Let's refer to the quantity h as the camera height. We can determine the camera height by substituting $x = 0$ into any of the equations of the lines $A'A$ and $B'B$:

$$h = \frac{lt}{l - a\sqrt{1+t^2}}(0 + a),$$

$$h = \pm \frac{al\sqrt{\left(\frac{a+b}{2ab}l\right)^2 - 1}}{l - a\frac{a+b}{2ab}l},$$

$$\boxed{h = \pm \frac{\sqrt{(a+b)^2l^2 - 4a^2b^2}}{b-a}}. \quad (2)$$

We take the camera height to be a positive number and adjust the sign of the slope of the line p accordingly. The slope is then positive if $b > a$, and negative if $b < a$.

4.1.3. Two-dimensional case

Now let's look at the full two-dimensional case (Figure 17). Here, the photographed object is a rectangle. Let us denote the vertices of the rectangle with A' , B' , C' and D' where A' is the lower left vertex and the vertices are denoted in clockwise order. Say there is a camera at point H and that the

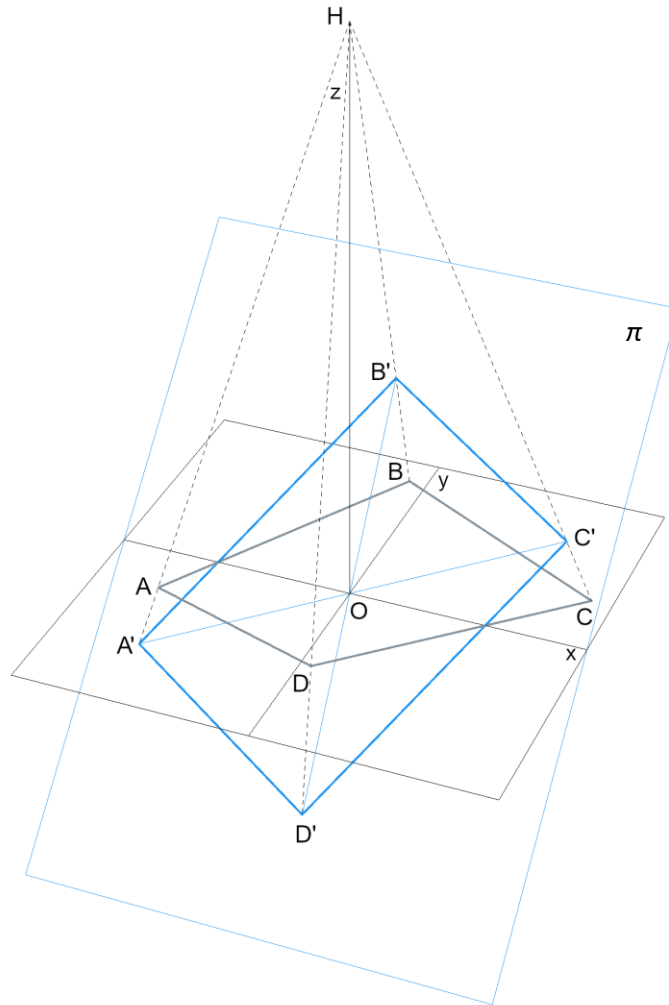


Figure 17: Two-dimensional case

rectangle $A'B'C'D'$ lies in the plane π . The point O is the intersection of the diagonals of the rectangle. Let's set define the coordinate system S so that the point O is the origin and the z -axis passes through H . Let us define point A as the intersection of the line HA' and the xy plane of the system S . Define points B , C and D analogously. When photographing the rectangle $A'B'C'D'$ with a camera in H , we get a photo that looks like a quadrilateral $ABCD$.

We use the following labels:

- a, b, c, d – the distances from points A, B, C, D to point O , respectively
- l – the distance from points A', B', C', D' to point O (this distance is the same for all of these points because $A'B'C'D'$ is a rectangle with O being its diagonal intersection)
- π – the plane in which the rectangle $A'B'C'D'$ lies
- h – z coordinate of point H (camera height).

In the two-dimensional case, the quantities a, b, c , and d are known (we can determine them from the coordinates of points A, B, C and D), but, as we will describe, l can be expressed using the quantities a, b, c and d , so it doesn't have to be known in advance as in the one-dimensional case. The goal is to determine the equation of the plane π in system S and camera height h .

We will determine the plane π as the plane in which both lines $A'C'$ and $B'D'$ lie. The equations of the lines AC and BD are known to us because we know the coordinates of points A, B, C and D . If we know the slope of the line $A'C'$ with respect to line AC and the slope of the line $B'D'$ with respect to line BD , we can determine the equations of the lines $A'C'$ and $B'D'$ in the system S and thus also determine the equation of the plane π .

For determining the slope of the line $A'C'$ with respect to the line AC , we can use the equation (1), substituting in a and c instead of a and b . Likewise, for determining the slope of the line $B'D'$ with respect to line BD , we can use the equation (1), substituting in b and d instead of a and b . The problem is that we don't know the quantity l .

Let us express l using the parameters a, b, c , and d . The camera height with respect to points A and C and with respect to points B and D should match. Therefore, according to equation (2):

$$h(a, c) = h(b, d),$$

$$\frac{\sqrt{(a+c)^2 l^2 - 4a^2 c^2}}{|c-a|} = \frac{\sqrt{(b+d)^2 l^2 - 4b^2 d^2}}{|d-b|}.$$

By squaring and rearranging, we obtain the equation:

$$\boxed{l^2 = \frac{a^2 c^2 (b-d)^2 - b^2 d^2 (a-c)^2}{(ab-cd)(bc-ad)}}. \quad (3)$$

Using equation (3) we can calculate l . Now we can use equation (1) to determine the slope of line $A'C'$ with respect to line AC , the slope of the line $B'D'$ with respect to line BD and, finally, the equation of plane π by finding the plane in which lines $A'C'$ and $B'D'$ lie. We can also determine h using equation (2). The coordinates of H are then given by $(0, 0, h)$.

The coordinates of A' can now be determined by finding the intersection of line HA with plane π . Analogously, we determine the coordinates of points B', C' and D' . The output image is represented by the rectangle $A'B'C'D'$. The width of the output image is the distance between points A' and D' , and the height of the output image is the distance between points A' and B' . Since we know the coordinates of points A', B' and D' in system S , we can translate the coordinates of each output image pixel into

system S coordinates. This means we can represent each pixel of the output image with a point in rectangle $A'B'C'D'$.

Furthermore, for each point T' of the rectangle $A'B'C'D'$ (which represents a pixel of the output image), we can find the corresponding point T of the rectangle $ABCD$ (pixel of the input image) as the intersection of the xy plane of the system S with the line HT' . So, for each pixel of the output image we find the corresponding pixel of the input image whose color values we copy to the output image.

Let's point out that we need to consider a few edge cases. If $a = b = c = d$, the dots already form a rectangle, so it is not necessary to carry out any transformation. Otherwise, if the denominator in equation (3) is zero, it is not possible to perform a unique transformation. In this case, the application notifies the user.

4.1.4. Implementation

The points are represented as three-dimensional vectors. Lines are implemented as a pair of vectors $(\vec{b}, \vec{d})$ representing the set of all points (vectors) that can be expressed as $\vec{b} + \lambda \vec{d}$ where λ is some scalar. Planes are implemented as an ordered pair of a vector and a scalar $(\vec{v}, b)$ representing the set of all points (vectors) $\vec{r}$ such that $\vec{v} \cdot \vec{r} = b$. In the code, all the necessary functions' implementations can be seen such as finding the intersection of two lines, the intersection of a line and a plane, a plane determined by two lines, etc.

The code that implements the described algorithm is located in `native/transform/quad_transform.cpp` file. The `QuadTransform::loadCoordinates` function loads the corner points of the area we want to straighten in counterclockwise order, with the lower-left point being first. The code determines the plane π (implemented as `fplane`) and the camera height h as described above. Define the projection of the point T being the intersection of line HT with the plane π (in the code, `putOnFloor` function calculates the projection of the point). The code determines the points A' , B' and D' in the system S as projections of points A , B and D . The height of the output image corresponds to the distance between points A' and B' , and the width of the distance between points A' and D' . In the output image's coordinate system, the origin is located at point A' (implemented as `newOrigin`), the x unit vector is $\widehat{A'D'}$ (implemented as `unitX`), and y unit vector is $\widehat{A'B'}$ (implemented as `unitY`).

In the `QuadTransform::process` function, the code iterates in parallel through all pixels of the output image. The `correspondingSrcCoors` function translates the coordinates of the point T' from output image's coordinate system to system S coordinates using the variables `newOrigin`, `unitX` and `unitY` and returns the corresponding point T of the input image calculated as the intersection of the line HT' and xy plane of the system S . In place of each pixel of the output image (`dst`), channel values of the corresponding pixel of the input image (`src`) are copied.

4.2. Book page straightening algorithm (see *Figures 8 and 9*)

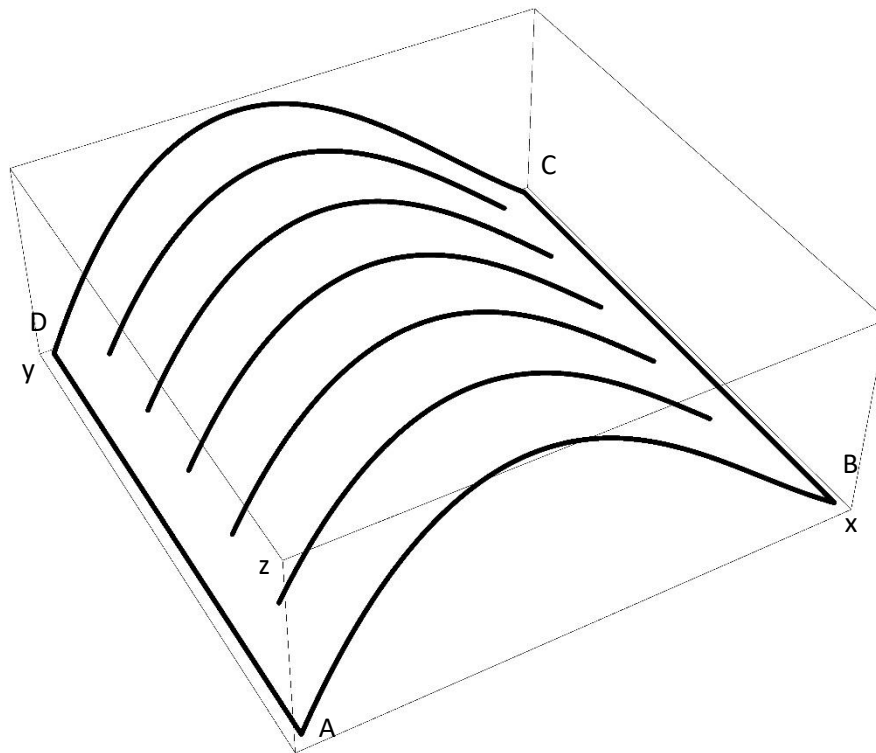


Figure 18: Book page model

Straightening a book page poses a challenge because the book page is not rectangular in shape because it is bent due to the binding. To be able to straighten a page, we need to make certain assumptions about its three-dimensional shape. The algorithm assumes the book page has the shape shown in *Figure 18*, that is, that the page can be placed inside a cuboid so that the two straight edges of the page correspond to the two edges of the cuboid, and the other two edges of the page are two parallel curves that rise vertically from the plane on which the book lies. Let's call this shape the book page "model". Model curve is a $z(x)$ function that describes the book page model.

4.2.1. Input and output data

Like in the quadrilateral transformation algorithm, the input consists of the image we want to transform and the coordinates of the corners of the book page. In addition to the above, the input also contains a list of points that define the curve of the edge of the page in the input image. This curve is later called the edge curve.

The output is an image of the straightened book page.

4.2.2. Determining the model curve from the edge curve

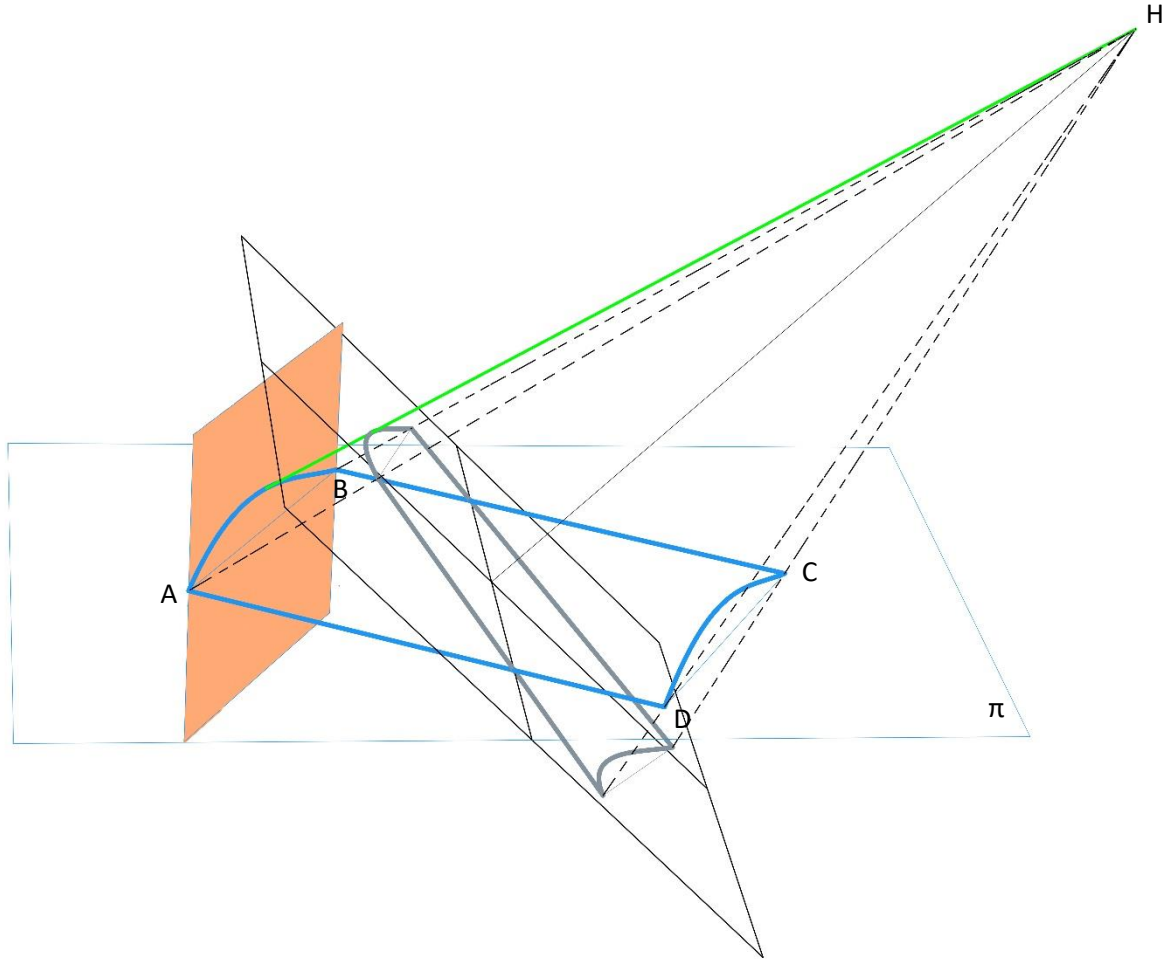


Figure 19: Determining the model curve from the edge curve

Figure 19 shows the plane in which the input image lies in black, and the edge of the book page in the input image is showed in gray. Point H represents the camera's oculous. Using the quadrilateral transformation algorithm described above, we can determine the position of point H , the plane π in which the book page model, shown in blue, lies, and the position of the vertices A , B , C and D of the book page model. The orange plane in Figure 19 is a plane perpendicular to plane π which contains line AB . Notice that plane π corresponds to the xy plane and the orange plane corresponds to the xz plane of the book page model (Figure 18). We determine the model curve $z(x)$ by projecting the points of the edge curve onto the orange plane as shown by the green line in Figure 19.

4.2.3. Transforming the image

Image transformation involves iterating through all the pixels of the output image and finding the corresponding pixels of the input image. In order to do this, for each point of the output image (straightened book page) we have to determine its position on the book page model (Figure 18). After determining the position on the model, we can draw a line between that point on the model and H to obtain the corresponding point, i.e. pixel of the input as the intersection of that line and the plane in which the initial image lies (Figure 20).

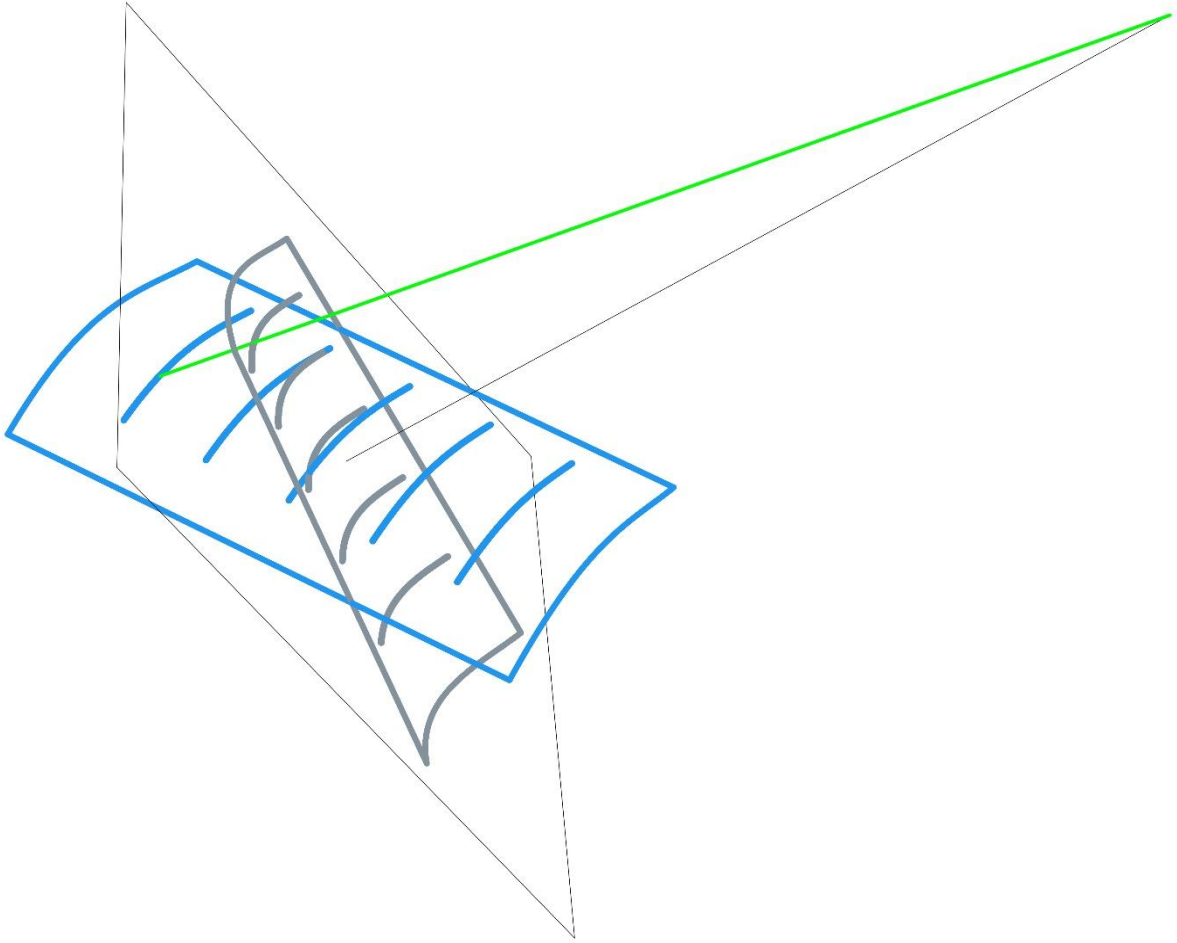


Figure 20: Finding the corresponding input image pixel (gray) for each output image pixel (blue)

The height of the final image is the length of the AD edge in the book page model (Figure 18), and the width is given by the length of the model curve $z(x)$ between points A and B . A point T of the output image with coordinates (x_T, y_T) has the coordinates $(x_T', y_T, z(x_T'))$ on the book page model such that the length of the $z(x)$ curve between the x -coordinate of point A and x_T' is equal to x_T . This way, for every pixel of the output image, we first determine its position on the book page model and, as shown in Figure 20, find its corresponding input image pixel.

4.2.4. Implementation

The code that implements the described algorithm is located in `native/transform/book_transform.cpp` file. The `BookTransform::loadCoordinates` function loads the corner points of the area we want to straighten in counterclockwise order, with the lower-left point being first. The function also loads a list of points that determine the edge curve of the page. The `curvePosition` parameter determines whether the edge curve is located between points $p1$ and $p2$ (*false*) or between points $p4$ and $p3$ (*true*). In the application, this parameter is always set to *false* because testing has determined that it is more convenient for the user to specify the curve at the top edge of the book page. The curve is linearly approximated between the points in the list which determine it. For a point T with x -coordinate x_T in the output image, its calculated x and z coordinates on the book page model are located in `xzValues` list at position x_T .

In the `BookTransform::process` function, the code iterates through all pixels of the output image (*dst*). For each pixel of the output image, its book page model coordinates (*modelX*, *y*, *modelZ*) are

determined and, as shown in *Figure 20*, the corresponding input image pixel is found. The color values of this input image pixel are copied to the output image pixel.

4.3. Interaction with external code

The external code passes the input image as a pointer to a list of bytes in which the image is stored as a bitmap, that is, for each pixel of the image at the position (x, y) , the value of the channel z is located at the index $y * s + x * c + z$ where c is the number of channels of the bitmap, and s is the so-called *stride*. For historical reasons, bitmaps can be formatted in two ways:

- *stride* is the product of the width of the image and the number of channels; bitmap isn't padded
- *stride* is the smallest number divisible by 4 that is greater than or equal to the product of the width of the image and the number of channels (this format is used in .bmp files); bitmap is padded.

4.3.1. Function calling order

1. At the beginning, the *Prepare* function is called, the first argument of which determines whether the input images are padded, and the second argument whether the output images should be padded. The default values are that no image is padded.
2. The *CanTransform* function is called with arguments that correspond to the coordinates of the vertices of the quadrilateral that we want to straighten. The function returns whether the necessary transformation can be performed.
3. The *LoadCoordinates* or *BookLoadCoordinates* function is called, depending on the transformation we want to perform. Both functions calculate the necessary values to perform the transformation and return whether it is possible to perform the necessary transformation.
4. The *GetWidth* and *GetHeight* functions can be called so that the external code knows what the width or height of the output image will be, respectively.
5. The *GetRequiredDstSize* function is called with the argument of how many bytes the image occupies per pixel (e.g., the RGB format occupies 3 bytes per pixel). The function returns how many bytes of memory the external code should allocate. The external code allocates the required memory.
6. The *Process* or *BookProcess* function is called, depending on the transformation we want to perform. The arguments of both functions are a pointer to the input bitmap (*src*), its dimensions and a pointer to the memory that the external code allocated in the previous step (*dst*). The function writes the output image bitmap to the memory location *dst* points to.

4.4. Testing

When implementing these algorithms, the code was regularly tested. Here is an example of testing the *Intersection* function defined in the *native/basic_linear/line.cpp* file, which returns the intersection point of two lines, and throws an exception if it does not exist or if there is no unique intersection. The

$A(x_A, y_A, z_A): (1, 1, 1)$
 $B(x_B, y_B, z_B): (-1, -1, -1)$
 $C(x_C, y_C, z_C): (-3, -6, -6)$
 $D(x_D, y_D, z_D): (-2, -1, -1)$

Calculate

Points on line 1:
 1 1 1
 -1 -1 -1
 Points on line 2:
 -3 -6 -6
 -2 -1 -1
 Intersection: -2.25 -2.25 -2.25

Point of Intersection
 $(x_0, y_0, z_0) = (-2.25, -2.25, -2.25)$

Figure 21: Intersection calculated using an Internet source (left) and the Intersection function (right)

function was tested by being called as $Intersection(Line(A, B), Line(C, D))$ where A, B, C and D were arbitrary three-dimensional points and the result of the function was compared with the result obtained here: https://www.analyzemath.com/Geometry_calculators/intersection-of-two-lines-in-3D-calculator.html (Figure 21).

5. Further development plan

One of the goals in further development would be to implement text recognition from photographed documents (OCR) with AI using libraries such as *pytesseract*³. A more ambitious goal in further development would include the ability to automatically recognize the corners of the photographed document. This could be achieved with the *OpenCV*⁴ library using the *Harris Corner Detection*⁵ technique. The first attempts at implementation have shown that a lot of testing and parameter adjustments will be required to achieve reliable automatic corner recognition.

6. Additional examples



Figure 22: Before the transformation



Figure 23: After the transformation

³ <https://pypi.org/project/pytesseract/>

⁴ <https://opencv.org/>

⁵ https://docs.opencv.org/4.x/dc/d0d/tutorial_py_features_harris.html



Figure 24: Before the transformation



Figure 25: After the transformation

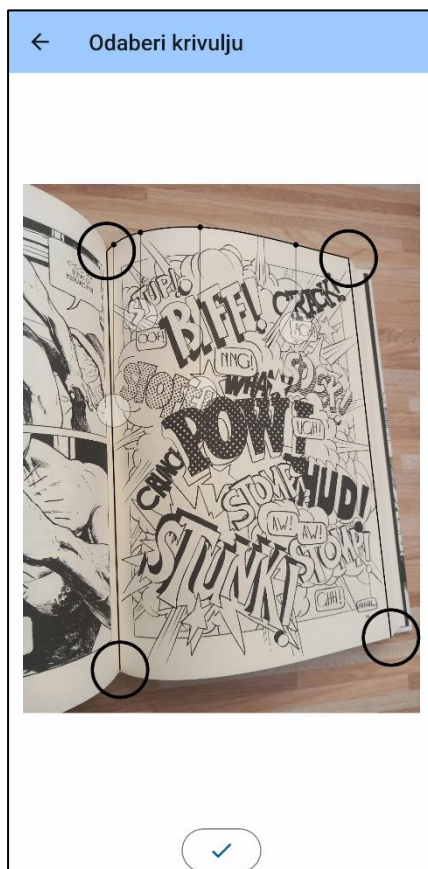


Figure 26: Before the transformation

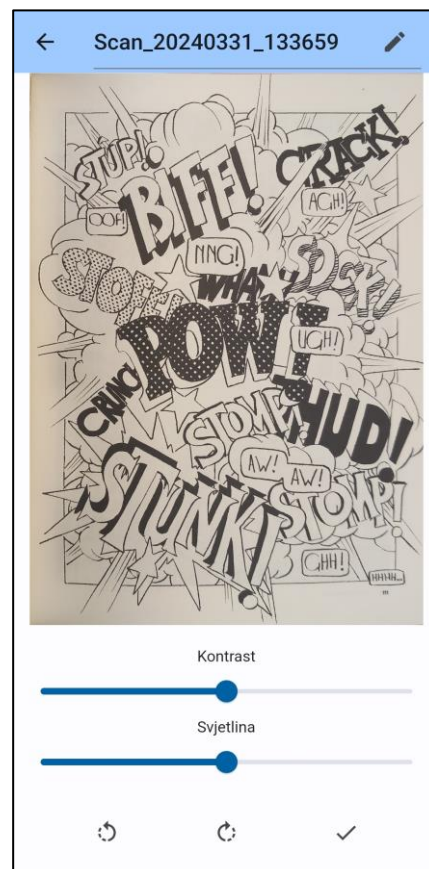


Figure 27: After the transformation